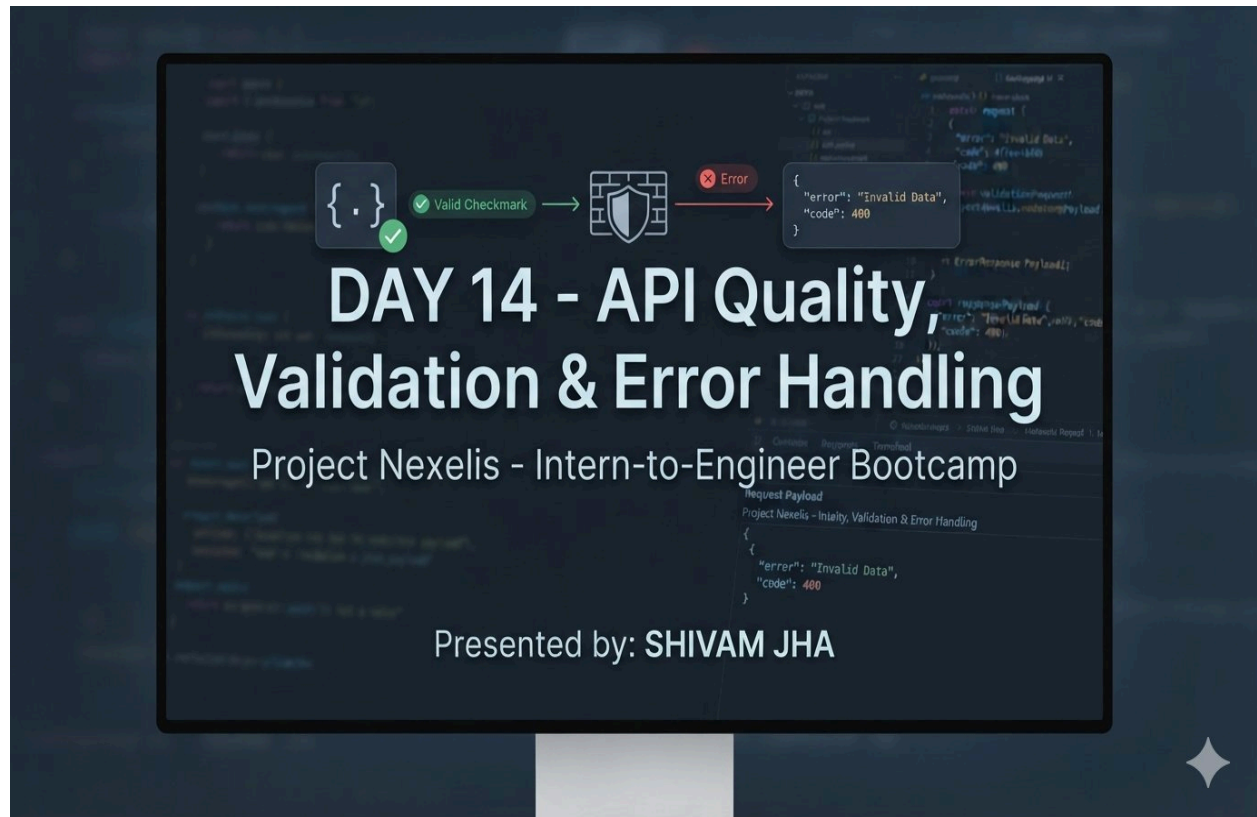




1: Day 14 Objective

Day 14 moves the Employee API from **functionally correct** to **production-quality**. It adds input validation, standardized errors, correct status codes, pagination, search, sorting and logging, so the API behaves predictably under both valid and invalid usage.

Key Points

Core principle

- An API is not complete because it works. It is complete when it is safe, predictable, consistent and debuggable.

Scope: what gets built

Area	Deliverable
Validation	Invalid email, missing name, invalid department, duplicate ID or email
Errors	One JSON error format for every failure

Status codes 201, 204, 400, 404, 409, 422, 500 used correctly

Pagination `GET /employees?page=1&limit=20`

Search `GET /employees?search=John`

Sorting `GET /employees?sort=name`

Observability Logging and global exception handling

Engineering qualities targeted

- **Reliability:** bad input never corrupts data or crashes the server.
- **Predictability:** same situation, same response shape, same status code.
- **Scalability:** large datasets are served in pages, not in one response.
- **Debuggability:** every failure leaves a log entry.

Deliverables

- Updated API
- Validation documentation
- Error response examples
- Test cases

My Day 13 API worked, but it wasn't production-ready. Day 14 was about closing that gap.

This shows you understand that working code and quality code are different things.

- **I made every error return one JSON structure.** The frontend then needs a single error handler instead of one per endpoint.
- **I added pagination with a maximum limit.** It protects the server from unbounded queries and keeps response time steady as data grows.
- **I treat validation, error handling and logging as features, not afterthoughts.** Each one has its own layer and its own tests.

2: Input Validation

Input validation verifies that request data meets defined rules (type, format, range, required fields, business constraints) before it reaches business logic or the database.

Key Points:-

Three-layer model

- **Schema layer (Pydantic):** structure and format. Types, required fields, email format, numeric ranges.
- **Service layer:** business rules that need a lookup. Department exists, email not already registered.
- **Database layer:** final integrity guarantee. `NOT NULL`, `UNIQUE`, `FOREIGN KEY`.

Rule placement

Rule	Layer	Status
Missing <code>name</code>	Pydantic	422
Invalid email format	Pydantic	422
<code>salary <= 0</code>	Pydantic	422
Department does not exist	Service	404
Duplicate email	Service + DB <code>UNIQUE</code>	409

Principles

- Fail early, before any DB call.
- Never trust client input.
- Normalize before comparing or storing (trim names, lowercase emails).
- The service check gives the friendly error. The DB constraint is the safety net.

Why It Is Used

- **Data integrity:** blocks garbage rows such as negative salaries and malformed emails.
- **Security:** shrinks the attack surface for malformed payloads.
- **Efficiency:** rejecting bad requests before a query saves DB work.
- **Developer experience:** clients receive field-level feedback.
- **Concurrency safety:** two simultaneous requests can both pass a service check. Only the `UNIQUE` constraint stops the second insert.

3: HTTP Status Codes

HTTP status codes are standardized three-digit numbers in every response that tell the client the outcome of its request. The first digit gives the category: 2xx success, 4xx client error, 5xx server error.

Key Points

Categories

- **2xx (Success):** the request worked.
- **4xx (Client error):** the client sent something wrong. Retrying the same request will fail again.
- **5xx (Server error):** the server failed. The request may be valid, and retrying later might work.

Codes used in this project

Code	Name	Use when
200	OK	Successful GET or PUT
201	Created	Successful POST that created a resource
204	No Content	Successful DELETE, with no body returned
400	Bad Request	Request is understandable but logically invalid (e.g., unknown sort field)
404	Not Found	Employee or department ID does not exist
409	Conflict	Request is valid but clashes with existing data (duplicate email)
422	Unprocessable Entity	Body or parameters fail Pydantic validation
500	Internal Server Error	Unexpected bug or failure

Decision rules

- Blame the client (4xx) only when the client is actually wrong.
- Never return 200 with an error message inside the body.
- Never return 500 for something the client caused.
- The same situation must always return the same code, on every endpoint.

Why It Is Used

- **Machine-readable outcome:** frontends, mobile apps and other services branch on the code, not on message text.
- **Retry logic:** clients retry 5xx but never retry 4xx.
- **Monitoring:** dashboards alert on a spike in 5xx and track 4xx to spot misuse.
- **Caching and tooling:** browsers, proxies and API gateways rely on the codes.
- **Shared language:** every developer already knows what 404 means.

4: Consistent Error Responses

A consistent error response is a single, fixed JSON structure returned for every failure in the API, whether it is a missing record, a validation failure, a conflict or a crash.

Key Points

Standard error envelope

```
json
{
  "success": false,
  "error": {
    "code": "EMPLOYEE_NOT_FOUND",
    "message": "Employee with id 9999 was not found",
    "details": null
  }
}
```

Field	Audience	Purpose
<code>success</code>	Client code	Quick check: <code>false</code> on every error
<code>error.code</code>	Client code	Stable machine-readable identifier. Never changes once published
<code>error.message</code>	Humans	Readable explanation. May be reworded over time
<code>error.details</code>	Client code	Optional extra data, e.g. field-level validation errors

Why It Is Used

- **One client handler:** the frontend writes a single error-handling function instead of one per endpoint.
- **Stable contracts:** clients switch on `code`, so rewording a `message` never breaks them.
- **Faster debugging:** support and developers can identify the exact failure from the code alone.
- **Security:** internal details stay in server logs and never reach the client.
- **Team consistency:** new endpoints inherit the pattern automatically.

Scenario Overview

A client calls the API and may hit a missing employee, an invalid payload, or an unknown URL. All three must come back in the same error envelope.

5: Exception Handling

Exception handling is the practice of catching runtime failures, both expected (missing record, duplicate) and unexpected (bug, database outage), and converting them into controlled responses instead of crashes or raw stack traces.

Key Points

Three classes of failure

Class	Example	Handled by	Result
Expected business error	Employee not found, duplicate email	<code>AppException</code>	404 / 409
Invalid input	Bad email, negative salary	<code>RequestValidationError</code> handler	422
Unexpected failure	Bug, DB connection lost	Global <code>Exception</code> handler	500

App Rules:-

- Catch specific exceptions first. Never use a bare `except:`.
- Always `db.rollback()` after a failed database operation, or the session stays in a broken state.
- Log the full traceback on the server. Send only a generic message to the client.
- Never swallow an exception silently (`except: pass`).
- Never return `str(exc)` on a 500. It can leak SQL, table names or file paths.

Where each concern lives

- **Service layer:** catches `IntegrityError`, rolls back, raises `AppException`.
- **Dependency (`get_db`):** guarantees rollback and session close, even on crash.
- **Global handlers:** last line of defence for anything nobody anticipated.

Why It Is Used

- **Availability:** one bad request must never take the server down.
- **Security:** raw tracebacks reveal internal structure to attackers.
- **Data safety:** rollback prevents half-finished writes and locked sessions.
- **Operations:** a logged traceback with request context turns a vague "it broke" into a findable bug.
- **User experience:** clients get a clean, actionable message instead of an unreadable failure.

Scenario Overview:-

An employee insert fails because of a race-condition duplicate, and separately an unexpected bug or database outage occurs. Both must return clean JSON, leave the database consistent and be logged.

- **I split failures into expected, invalid and unexpected, and each gets its own handler.** Business errors keep their specific status. Anything unforeseen falls through to a catch-all that returns a safe 500.
- **The client gets a generic message, and the log gets the full traceback.** This protects internal details from users and still gives me everything needed to debug.
- **I roll back on every failure.** Without a rollback, a failed commit leaves the session unusable and can hold partial data. `get_db` guarantees the cleanup in one place.
- **I wrapped the commit and conflict logic in one helper, `commit_or_conflict`.** Every service handles duplicates the same way, so there is no copy-pasted `try/except` to get out of sync.

6: Pagination

Pagination splits a large result set into fixed-size pages, controlled by `page` and `limit` query parameters, and returns metadata so the client can navigate between pages.

Key Points

Request contract

http

GET /api/employees?page=2&limit=20

Parameter	Default	Rule	Violation
page	1	Integer, >= 1	422
limit	20	Integer, 1 to 100	422

}

Rules

- Always enforce a **maximum limit**. An unbounded limit defeats the purpose.
- Always use a **deterministic ORDER BY**. Without one, PostgreSQL may return rows in a different order per query, so records repeat or vanish across pages.
- A page beyond the last returns **200 with an empty data list**, not 404. The collection exists, and it just has nothing on that page.
- Filters (Page 7) must apply **before** counting, so **total** matches what the client sees.

Why It Is Used

- **Performance**: the database and network move 20 rows, not 100,000.
- **Memory safety**: the server never loads the entire table into RAM.
- **Scalability**: response time stays flat as data grows.
- **Usability**: the frontend can render page controls from **meta**.
- **Abuse protection**: the **limit** cap stops a client from requesting the whole table in one call.

7: Filtering & Search

Filtering narrows a result set to records that match given conditions.

Search is a filter that matches text (here, a partial, case-insensitive match on name or email), applied through query parameters.

Key Points

Request contract

http

GET /api/employees?search=john

GET /api/employees?department_id=2

GET /api/employees?search=john&department_id=2&page=1&limit=20

Parameter	Type	Behavior
search	string, 1 to 100 chars	Case-insensitive "contains" match on name or email
department_id	integer, >= 1	Exact match on the department

Why It Is Used

- **Usability:** nobody scrolls through 5,000 employees to find one person.
- **Efficiency:** the database filters, so only relevant rows cross the network.
- **Composability:** search, department filter, sorting and pagination combine freely.
- **Security:** parameter binding and input escaping stop injection and accidental wildcard abuse.

8: Sorting

Sorting orders the result set by a client-chosen field and direction, controlled by a **sort** query parameter. A **-** prefix means descending.

Key Points

Request contract

http

GET /api/employees?sort=name # A to Z (ascending)

GET /api/employees?sort=-salary # highest salary first (descending)

GET /api/employees?search=john&sort=-name&page=1&limit=20

Parameter	Default	Rule	Violation
sort	name	Must be in the whitelist, optional - prefix	400 INVALID_SORT_FIELD

Core mechanics

- The client sends a **field name**, and the server maps it to a **column object** through a dictionary. Client text never touches SQL.
- - prefix selects `.desc()`, otherwise `.asc()`.
- Sorting is applied **before** `OFFSET / LIMIT`. The `COUNT` is unaffected.
- A **tie-breaker** (`employee_id`) is always appended as the last sort key.

Why It Is Used

- **Usability**: "highest paid first" and "A to Z" are among the most common list needs.
- **Correct pagination**: a stable, fully determined order is required for pages to be consistent.
- **Security**: a whitelist limits what clients can sort on and blocks injection through the parameter.
- **Consistency**: one reusable helper gives every list endpoint the same `sort` behavior.

Scenario Overview

An HR manager wants employees listed by highest salary, with ties in a stable order, and a clear error if the client asks for a field that isn't sortable.

9: Logging

Logging records timestamped, severity-tagged events (requests, business actions, warnings, failures) so that developers can reconstruct what happened after the fact.

Key Points

Log levels

Level	Use for	Project example
DEBUG	Detailed diagnostics, off in production	Query parameters received
INFO	Normal, meaningful events	Employee created id=42
WARNING	Unusual but handled	Duplicate email attempt, 4xx response
ERROR	Operation failed	Unhandled exception, DB failure
CRITICAL	System unusable	Cannot connect to the database at startup

What a good log line contains

- **When:** timestamp
- **How bad:** level
- **Which request:** request ID (ties every line of one request together)
- **Where:** logger name (module)
- **What:** short message with IDs, not personal data

Why It Is Used:-

- **Debugging in production:** you can't attach a debugger to a live server. Logs are the evidence.
- **Traceability:** the request ID lets you pull every line of one failing request out of thousands.
- **Monitoring:** rising 4xx or 5xx counts, or slow request times, show up in logs before users complain.
- **Auditing:** "who created or deleted this employee, and when" is answerable.
- **Security:** repeated failures can reveal abuse. Careless logging can also leak data, hence the rules above.

Scenario Overview

An employee creation fails in production and the client reports "it didn't work". You need to trace that exact request, see what the service decided, and see the traceback, without exposing internals to the client.

10: API Consistency

API consistency means every endpoint follows the same conventions for naming, URL structure, request parameters, success shape, error shape and status codes, so that once a client learns one endpoint, it can predict all the others.

Key Points

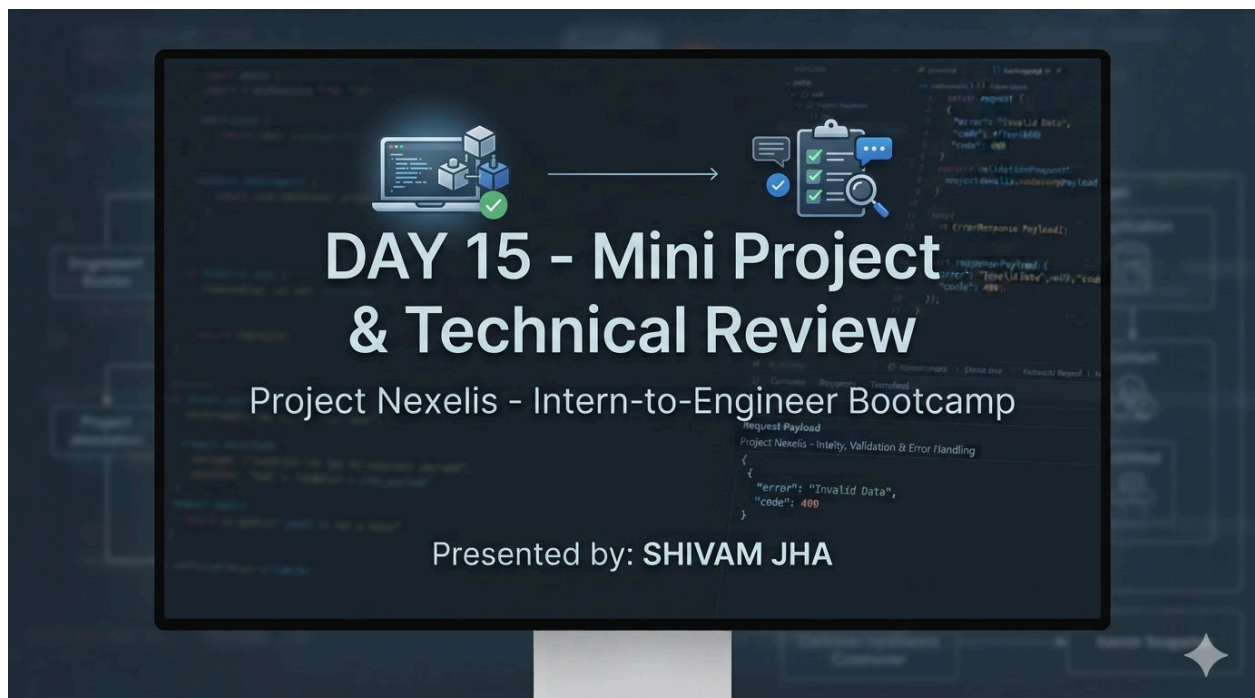
Conventions adopted in this project

Area	Convention
URLs	Plural nouns under a prefix: <code>/api/employees</code> , <code>/api/departments</code>
HTTP verbs	<code>GET</code> read, <code>POST</code> create, <code>PUT</code> update, <code>DELETE</code> remove. Never verbs in URLs (<code>/getEmployee</code>)
Field names	<code>snake_case</code> everywhere
Query parameters	<code>page</code> , <code>limit</code> , <code>search</code> , <code>sort</code> , plus <code><resource>_id</code> filters
Single-resource success	<code>{"success": true, "data": {...}}</code>

List success	<pre>{"success": true, "data": [...], "meta": {...}}</pre>
Error	<pre>{"success": false, "error": {"code", "message", "details"}}</pre>
Delete success	204, empty body (the one intentional exception)
Status codes	Same situation, same code, on every endpoint

Why It Is Used

- **Client simplicity:** one success parser and one error parser cover the entire API.
- **Faster onboarding:** a new developer learns the pattern once.
- **Fewer bugs:** clients never need special cases such as "this endpoint returns a bare list".
- **Maintainability:** changing the envelope means editing one class, not twenty endpoints.
- **Professional trust:** predictable APIs signal engineering quality.



1. Problem Solving:-

Question:-

What was the most difficult problem you faced, and how did you solve it?

The most difficult problem I faced was **integrating the FastAPI application with PostgreSQL while maintaining a clean API structure and handling validation and database errors correctly.**

During the earlier days, I first built the Employee API using **FastAPI, Pydantic, and an in-memory data structure.** When moving toward PostgreSQL integration, I had to understand how the API layer, validation layer, service/repository layer, SQLAlchemy models, and PostgreSQL database work together.

I also encountered a database schema mismatch where the SQLAlchemy query expected an **id** column that was not present in the existing **employees** table. I traced the error from the API request → service layer → repository → SQLAlchemy → PostgreSQL and identified that the **application model and database schema were inconsistent.** I resolved it by checking the actual database structure and aligning the SQLAlchemy model and PostgreSQL table definition.

Problem-Solving Approach



What I Learned

- Read the complete error traceback instead of changing code randomly.
- Trace an error through each application layer.
- Keep database models and application models synchronized.
- Use validation before database operations.
- Test APIs through Swagger and verify database results independently.
- Separate **API logic**, **business logic**, and **database logic** for easier debugging.

The major challenge was not writing the API itself, but integrating multiple layers correctly. I learned to trace problems from the API request down to the database instead of treating the error as an isolated code issue.

2. Self-Review

Question

If you had another two days, what would you improve?

If I had another two days, I would focus on improving the **production readiness, testing, and usability** of the Employee Management System rather than adding unnecessary features.

Improvements I Would Make

1. Automated Testing

I would expand the test suite using `pytest` to cover:

- Employee creation
- Employee retrieval
- Search
- Update
- Delete
- Invalid email
- Negative salary

- Missing required fields
- Duplicate email
- Non-existing employee

2. Better Database Management

I would introduce proper **database migrations** so that future schema changes can be safely applied without manually modifying the database.

3. API Quality

I would further improve:

- Consistent error responses
- Logging
- Pagination
- Filtering
- Sorting
- Input validation
- API documentation

These areas directly extend the API-quality work from Day 14.

4. Frontend Integration

After stabilizing the backend, I would build a simple React frontend containing:

Dashboard



Employee List



Search / Filter



Add / Edit / Delete Employee



Department Management

The frontend would communicate with the existing FastAPI REST API rather than directly accessing PostgreSQL.

5. Security & Production Improvements

I would also consider:

- Authentication and authorization
- Better environment configuration
- CORS configuration
- Centralized exception handling
- API-level security improvements

With two additional days, I would prioritize quality over adding more features. My focus would be automated testing, database migrations, stronger error handling, and a clean React frontend on top of the stable API.